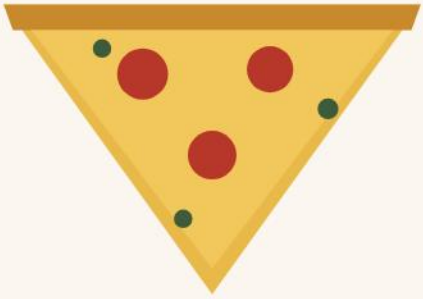
PROYECTO DE PROGRAMACIÓN · MEMORIA

Caja registradora de una pizzería

Desarrollado en Python

Versión 1 (funciones) y versión 2 (clases)





INTEGRANTES

Alex Rodríguez
Pau Mas

FECHA
Oct. – nov. de 2026

HERRAMIENTAS
Python · GitHub · Trello

REPOSITORIO
Puuaa7/ProyectoCajaRegistradora

Memoria del proyecto PIZZA

Proyecto de Programación · 22 de octubre de 2026 · **Actualizada el 13 de noviembre de 2026 con la versión 2**

La versión 1 (octubre) está hecha con funciones y la versión 2 (noviembre) con programación orientada a objetos; al arrancar `main.py` se elige cuál usar. En cada apartado va primero lo de la versión 1 y después un bloque «Versión 2».

1. Integrantes y datos del proyecto

Integrante	Usuario en GitHub	Iniciales en Trello
Alex Rodríguez	alexrb29	AR
Pau Mas	Puuaa7	PR

- **Proyecto:** caja registradora de una pizzería en Python.
- **Repositorio:** github.com/Puuaa7/ProyectoCajaRegistradora
- **Tablero:** Trello, «Caja Registradora - Pizzería».
- **Link Trello:** https://trello.com/b/nt1k8GgM/%F0%9F%8D%95-caja-registradora-pizzeria
- **Tablero de la versión 2:** Trello, «Caja Registradora - Pizzería (POO)»: https://trello.com/b/6aeb7b4b2dc99e7f30ca0fd1
- **Rama de la versión 2:** `version2` (ver en GitHub)

2. Distribución de tareas y responsabilidades

Nos repartimos el trabajo por partes del programa: Pau se encargó sobre todo de los cálculos, las ofertas y el main. Alex del carrito, el ticket, las bebidas, las pruebas y el README. Cada tarea es una tarjeta de Trello con responsable, descripción, lista de comprobación y estado.

"Las tareas que ponemos pendientes es porque a día de hoy 22/10/26 que estamos trabajando en ellas y avanzando en la memoria a la vez, el día de la entrega ya estarán todas terminadas"

Tarea (tarjeta de Trello)	Responsable	Estado a 22/10
Revisar gastos fijos y temporadas	Pau	Terminado
Comprobar cálculo de precios e IVA	Pau	Terminado
Programar 4 ofertas	Pau	Terminado
Aplicar las ofertas al carrito	Pau	Terminado
Crear bebidas y precios	Alex	Terminado
Crear carrito de compra	Alex	Terminado
Calcular total del carrito e IVA	Alex	Terminado
Crear ticket de compra	Alex	Terminado
README.md	Alex	Terminado

Tarea (tarjeta de Trello)	Responsable	Estado a 22/10
README (cambios del 22/10)	Alex	En revisión
Cálculo de precio por temporada	Pau	En proceso
Pruebas del programa (tests/)	Alex	En proceso
Capturas de Trello para la memoria	Pau	En proceso
Memoria del proyecto (PDF)	Pau	En proceso
Repositorio GitHub y commits	Los dos	En proceso
Mostrar los precios con 2 decimales	Alex	Pendiente
Controlar cantidades no válidas	Pau	Pendiente
Oferta de temporada	Pau	Pendiente
Revisión final y entrega	Los dos	Pendiente

En GitHub cada parte tiene su rama (`feature/calculos`, `feature/pizzas`, `feature/ofertas`, `feature/aplicar-ofertas`, `feature/bebidas`, `feature/ticket`, `feature/menu`, `test/pruebas`, `docs/readme` y `docs/memoria`), que se une a `main` con un merge.

Versión 2

Para la versión 2 hicimos un tablero nuevo con una tarjeta por clase. El reparto de Trello es el de la tabla; en GitHub, la mayoría de las ramas de la versión 2 las subió Alex y la de la temporada, Pau.

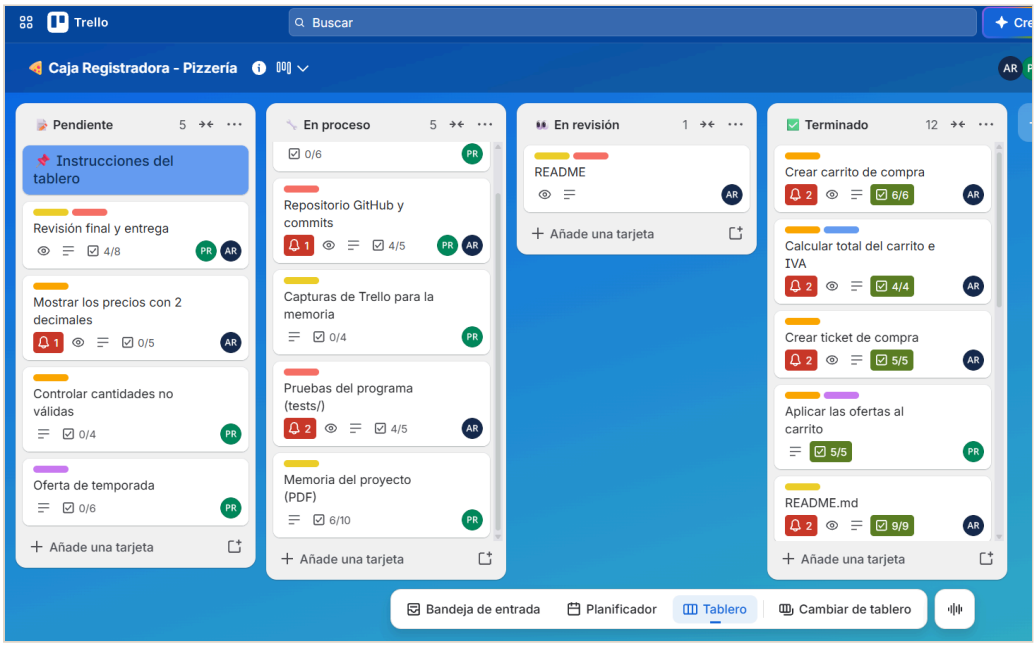
Tarea (tarjeta de Trello)	Responsable en Trello	Estado a 13/11
Acordar el diseño de clases	Pau	Terminado
Crear la estructura del proyecto en módulos	Pau	Terminado
Clase Ingrediente y catálogo de ingredientes	Pau	Terminado
Clase Negocio: gastos, IVA, margen y temporada	Pau	Terminado
Clase base Producto	Pau	Terminado
Clase Pizza (hereda de Producto)	Pau	Terminado
Completar la carta con 20+ pizzas	Pau	Terminado
Clase Bebida y criterio de precio	Alex	Terminado
Clase Menu (pizza + bebida y menú familiar)	Alex	Terminado
Clases Carrito y LineaCarrito	Alex	Terminado
Jerarquía de ofertas (clase Oferta y subclases)	Alex	Terminado
Clase Ticket: base imponible, IVA y total	Pau	Terminado
Clase Caja y menú principal (main.py)	Pau	Terminado
Validaciones y excepciones propias	Alex	Terminado

Tarea (tarjeta de Trello)	Responsable en Trello	Estado a 13/11
Pruebas con unittest (tests/)	Alex	Terminado
Repositorio GitHub, ramas y commits	Pau	Terminado
README.md	Alex	Terminado
Capturas de Trello para la memoria	Alex	Terminado
Memoria del proyecto (PDF)	Pau	Terminado
Revisión final y entrega	Los dos	En proceso

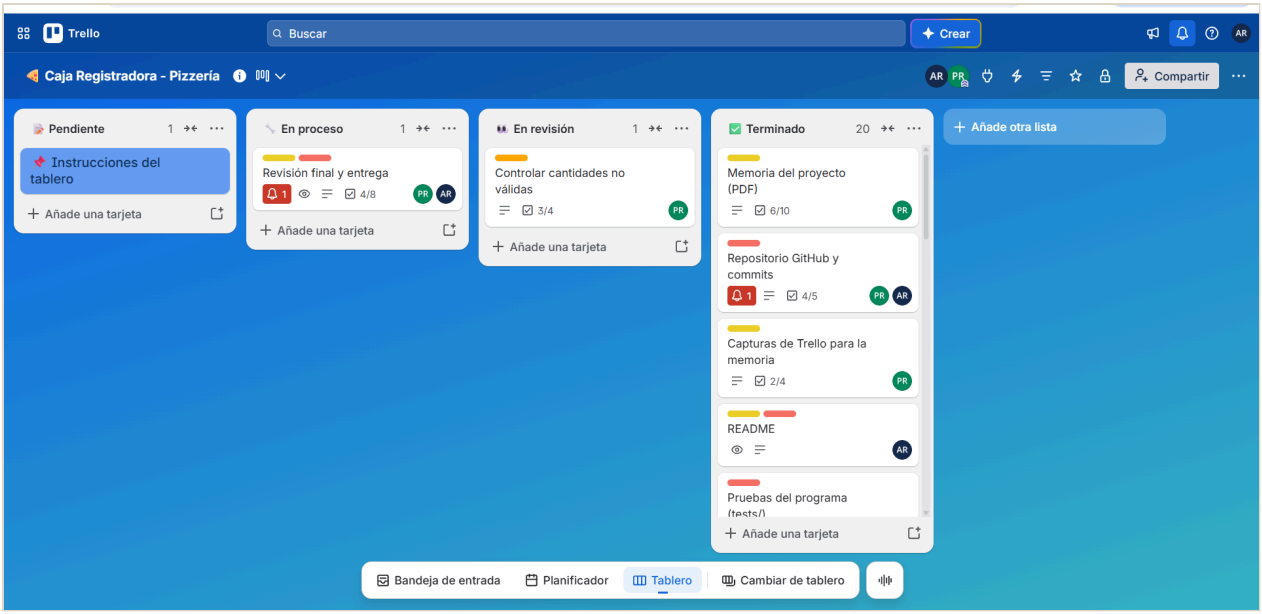
En GitHub la versión 2 vive en la rama `version2`. Cada clase se hizo en su propia rama (`version2_src_ticket`, `version2_src_calculos`, `version2_src_temporada`, `version2_src_ingredientes`, `version2_src_pizzas`, `version2_src_bebidas`, `version2_src_carrito`, `version2_src_ofertas`, `version2_src_menu`, `version2_src_gastos`, `version2_src_margenes`, `version2_src_excepciones`, `version2_src_producto`, `version2_src_lineacarrito`, `version2_src_menus` y `version2_src_jerarquia_ofertas`), más `version2_main`, `version2_tests`, `version2_readme` y `version2_docs`. Cada rama se junta con `version2` en cuanto está terminada; a `main` solo llegó, por error, la rama del ticket (ver apartado 6).

3. Tablero de Trello

El tablero tiene cuatro columnas: Pendiente, En proceso, En revisión y Terminado. Cada tarjeta lleva una etiqueta de color por tipo de tarea, la lista de pasos y la persona responsable (AR = Alex, PR = Pau).



Tablero de Trello de la versión 1 a 22/10/2026, columna Terminado.

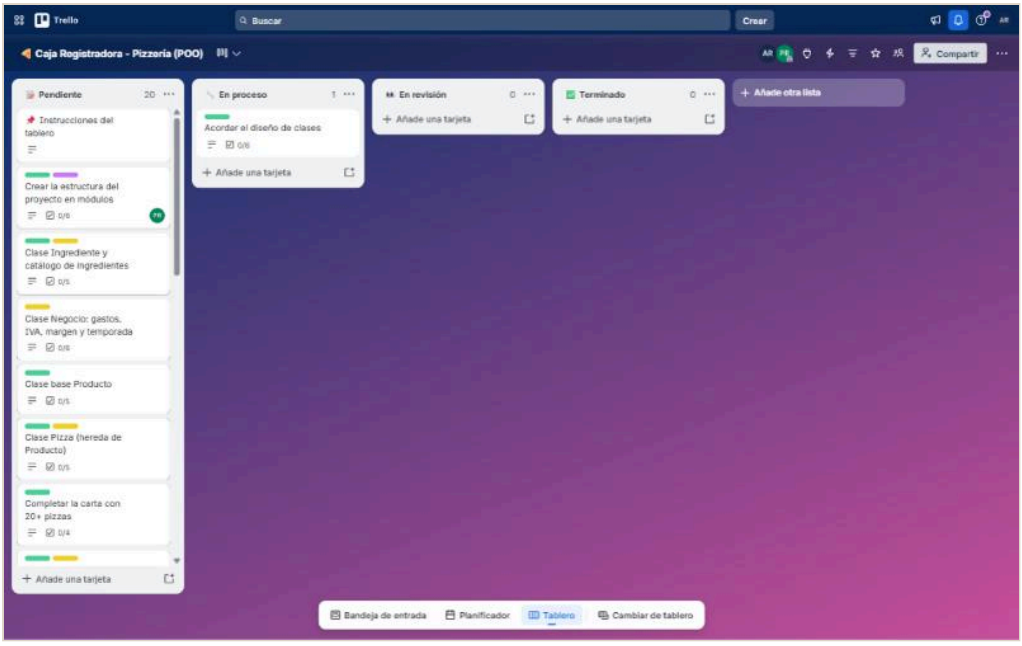


Tablero de Trello de la versión 1 a 22/10/2026.

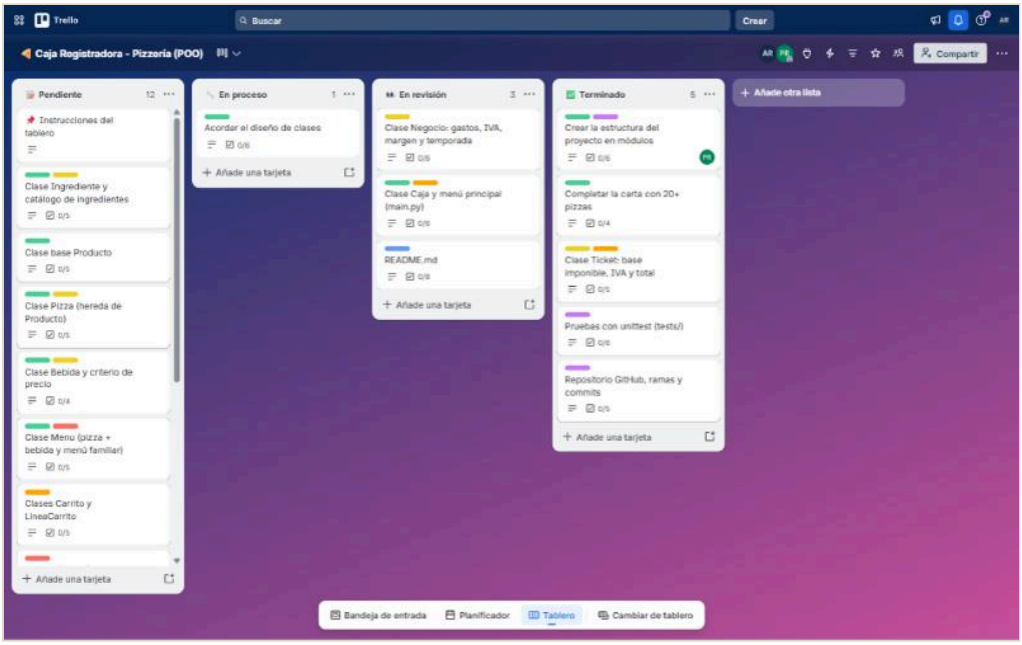
Versión 2

El tablero de la versión 2 («Caja Registradora - Pizzería (POO)») tiene las mismas cuatro columnas y una tarjeta por clase. El 13/11 vimos que casi todas las tarjetas seguían en «Pendiente» aunque mucho código ya estaba hecho, así que comparamos la lista de pasos de cada tarjeta con lo que había en `srcV2` y las movimos a su columna. Las tres primeras capturas muestran ese proceso; la cuarta, el tablero al terminar la versión 2.

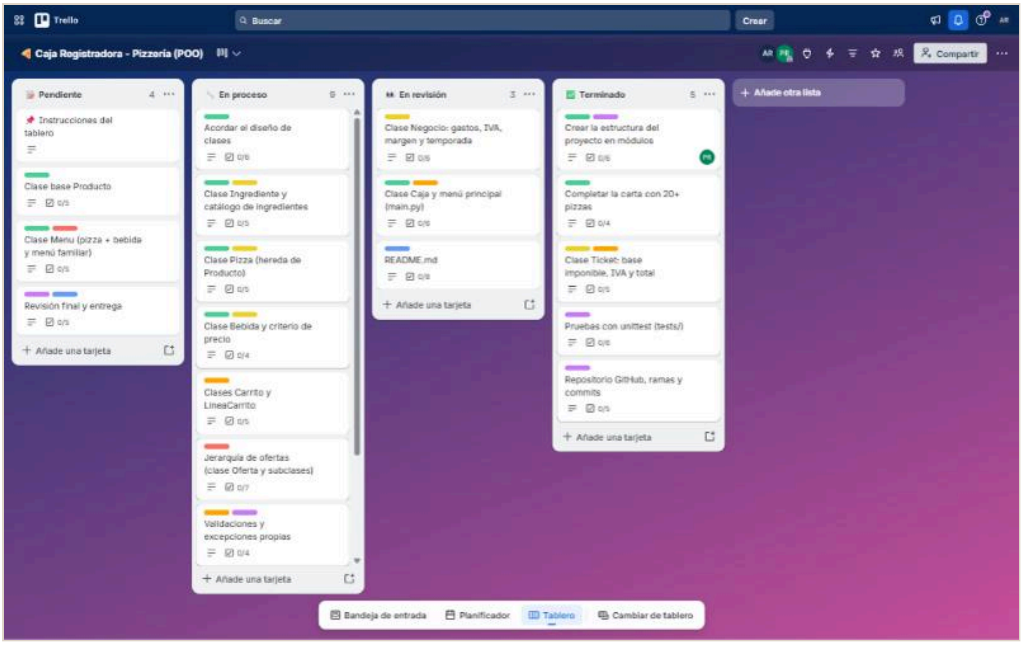
1. **Inicio:** las 20 tarjetas siguen en «Pendiente».
2. **A mitad:** ya están en su sitio las 5 tareas terminadas y las 3 que estaban en revisión.
3. **Clasificación terminada:** 8 tarjetas en proceso y 3 sin empezar (Producto, Menu y la revisión final), que eran justo las que pedían herencia y menús.
4. **Final:** después de hacer las clases que faltaban, todas las tarjetas en «Terminado» menos la revisión final y la entrega.



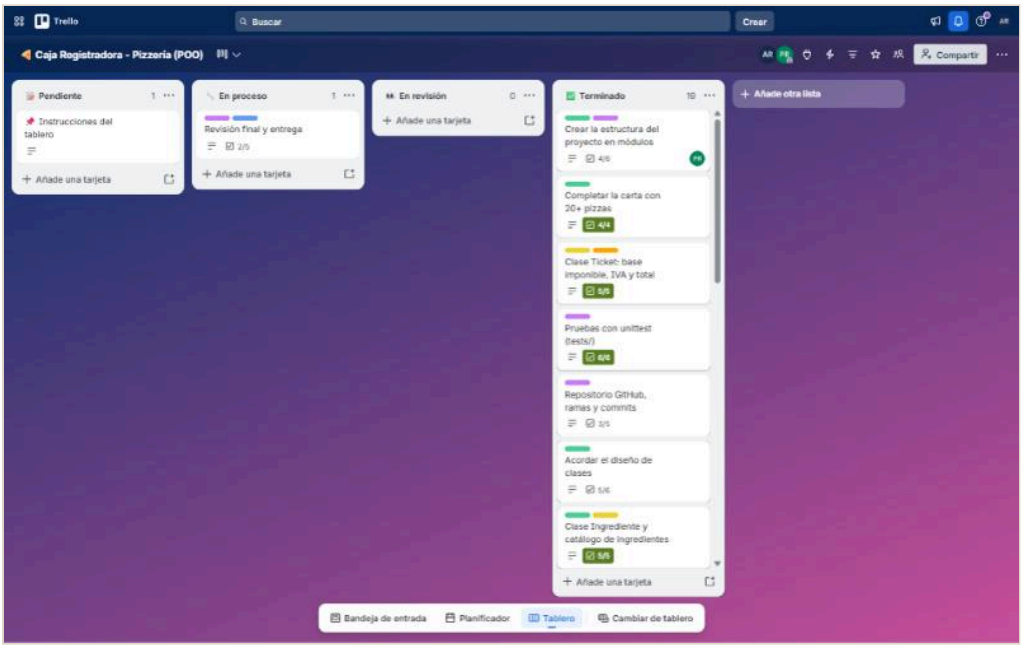
Captura 1 · 13/11/2026 · Inicio: las 20 tarjetas en «Pendiente».



Captura 2 · 13/11/2026 · A mitad: 5 tarjetas en «Terminado» y 3 en «En revisión».



Captura 3 · 13/11/2026 · Clasificación terminada: 9 en proceso (con «Acordar el diseño de clases»), 3 en revisión, 5 terminadas y 3 sin empezar.



Captura 4 · 13/11/2026 · Final: 19 tarjetas en «Terminado» con su lista de pasos y la revisión final en proceso.

4. Reuniones y acuerdos

Nos reunimos en cuatro sesiones de trabajo, que coinciden con los días en que hay commits en GitHub.

Fecha	Qué se trabajó	Acuerdos
22/10/2026	Cambios en el README y memoria	Cada cambio nuevo va en su propia rama y se sube en cuanto está terminado.

Fecha	Qué se trabajó	Acuerdos
21/10/2026	Pasar el código a ramas y unirlas a `main`	Una rama por parte del programa, con su responsable de Trello. Unir primero cálculos, pizzas, carrito y ofertas, y después bebidas, ticket, menú, pruebas y README.
14/10/2026	Reorganizar el proyecto en `src/`, `docs/` y `tests/`; bebidas, carrito y oferta 2x1	Usar la estructura `src/` y guardar el código antiguo en `z.functions_antiguo`.
07/10/2026	Estructura inicial, ingredientes, pizzas y gastos	Calcular el coste de cada pizza con el precio por kg de cada ingrediente y los gramos que lleva. Repartir las tareas en Trello.

Versión 2

Las sesiones de la versión 2 también coinciden con los días con commits en la rama `version2`.

Fecha	Qué se trabajó	Acuerdos
13/11/2026	Revisar el tablero de Trello; clase base Producto, LineaCarrito, Menu, jerarquía de ofertas y excepciones propias; README y memoria	Clasificar las tarjetas según lo que ya estaba hecho y completar el diseño orientado a objetos antes de cerrar la memoria.
10/11/2026	Pizza, Bebida, Carrito, Ofertas, Caja, `main.py` para elegir versión, pruebas, Gastos, Margen, README y log	Una rama por clase, que se junta con `version2` en cuanto está lista. A `main` no se pasa nada hasta revisar la versión 2.
09/11/2026	Ticket, Calculadora, Temporada (Pau) e Ingrediente	Hacer `git pull` de `version2` antes de subirla, para no pisar el trabajo del otro.
04/11/2026	Arreglar los imports de la versión 1 tras moverla a `srcV1`	La versión 1 se queda como está y tiene que seguir funcionando.
02/11/2026	Preparar el repositorio para dos versiones (`srcV1`, `srcV2` y la rama `version2`)	Hacer la versión 2 al lado de la 1 y elegir cuál usar al arrancar.

5. Decisiones importantes

La decisión principal fue calcular el precio de cada pizza a partir de sus ingredientes reales, en lugar de inventar precios de carta.

Decisión	Motivo
Precio de los ingredientes por kg y gramos de cada ingrediente por pizza	Así el coste depende de lo que lleva cada pizza de verdad.
21 pizzas en un diccionario de diccionarios (`pizza → ingrediente → gramos`)	El enunciado pide al menos 20 y así se recorren fácil con un bucle.
Margen de beneficio del 35% (multiplicar por 1,35)	Es el del enunciado y da precios de entre 11,95 y 17,36 € en temporada media.
Bebidas con precio fijo de carta, IVA incluido	Solo se compran y se venden.

Decisión	Motivo
El IVA se separa al final, sobre el total con descuentos	Así el IVA se calcula sobre lo que paga de verdad el cliente.
Estructura de carpetas `src/`, `docs/` y `tests/`	Es más ordenada y permite importar con `from src...`.
El día de la semana se coge de la fecha del ordenador	Para que la oferta del martes funcione sola; se puede cambiar en el menú para probarla.
Usar listas, diccionarios, tuplas y conjuntos	Carrito = lista, carta = diccionario, temporadas y días = tuplas, pizzas del 2x1 = conjunto.

Versión 2

La decisión principal fue pasar el programa a clases sin cambiar los precios: las pruebas comparan la versión 2 con la 1 y dan lo mismo.

Decisión	Motivo
Mantener la versión 1 y elegir la versión al arrancar `main.py`	Se pueden comparar las dos y la 1 sigue funcionando.
Clases abstractas `Producto` y `Oferta` (con `abc`)	No se pueden crear directamente y obligan a cada subclase a escribir su precio o su descuento.
`Pizza`, `Bebida` y `Menu` heredan de `Producto`	El carrito y el ticket tratan igual a todos los productos (polimorfismo).
Composición: la pizza tiene objetos `Ingrediente`, el carrito tiene `LineaCarrito` y la `Calculadora` usa `Gastos`, `Margen` y `Temporada`	Cada clase hace una sola cosa y los números se cambian sin tocar el código.
La «clase Negocio» de Trello se reparte en `Gastos`, `Margen`, `Temporada` y `Calculadora`	Son cuatro clases pequeñas más fáciles de probar que una grande.
La bebida mantiene el precio fijo de carta con IVA, no «precio de compra + margen» como ponía la tarjeta	Así las bebidas cuestan lo mismo en las dos versiones.
Menús como producto: pizza + bebida (1 € menos) y familiar, 2 + 2 (5 € menos)	Lo pedía la tarjeta de Trello. Sus pizzas no cuentan para las otras ofertas, para no descontar dos veces.
Oferta nueva: temporada baja, 10% en bebidas	La tarjeta pedía una oferta de temporada; en baja se venden menos pizzas.
Excepciones propias que heredan de `PizzeriaError`	La caja las captura todas con un solo `except` y no se cierra.
Atributos con `_` y `@property`	El nombre no se puede cambiar y los precios y cantidades se validan.

Así quedan las clases de la versión 2:

```
Herencia:

Producto (abstracta)        Oferta (abstracta)          PizzeriaError (Exception)
 ├── Pizza                   ├── Oferta2x1               ├── ProductoNoEncontradoError
 ├── Bebida                  ├── OfertaMenu              ├── IngredienteNoEncontradoError
 └── Menu                    ├── OfertaDiaSemana         ├── CantidadNoValidaError
                             ├── OfertaTemporada         └── PrecioNoValidoError
                             └── OfertaPedidoGrande

Composición (un objeto tiene o usa otros):

Pizza ──> Ingrediente              Menu ──> Producto (sus pizzas y bebidas)
Carrito ──> LineaCarrito ──> Producto
Calculadora ──> Gastos, Margen y Temporada
Caja ──> Carrito, Ofertas y Ticket
```

6. Errores encontrados y cómo se resolvieron

Error	Causa	Solución
`calcularPrecioPizza` fallaba al ejecutarse	Usaba una variable `total` que no existía	Usar la variable correcta, `precioTotalPizza`.
`ModuleNotFoundError: No module named 'Ofertas'`	Se ejecutaba `ticket.py` en vez del archivo principal	Ejecutar siempre `main.py`, que es el que arranca el programa.
`main.py` salía vacío en VS Code	Había una pestaña abierta con cambios sin guardar de una versión antigua	Cerrar la pestaña sin guardar y volver a abrir el archivo.
El 2x1 solo funcionaba con dos pizzas iguales	Se contaba cada pizza por separado	Juntar los precios de todas las pizzas de la oferta, ordenarlos y regalar la más barata de cada pareja.
«margarita » con un espacio al final no se encontraba	No se quitaban los espacios	Añadir `.strip()` al buscar el producto.
Las ofertas no se veían en el carrito, solo en el ticket	El carrito mostraba solo el total sin ofertas	Nueva función `mostrarDescuentosCarrito`, usada en el carrito y en «Ver ofertas».
La carpeta del proyecto no estaba conectada a Git	Se había descargado de GitHub como ZIP	Clonar el repositorio con `git clone` y trabajar siempre en esa carpeta.
Al unir ramas, `main` dejaba de funcionar	El menú y el ticket usaban funciones de ramas que aún no estaban unidas	Unir las ramas en orden: primero las de las que dependen las demás.
Las cinco ramas de Alex se subieron a GitHub a la vez	Se preparó todo fuera del repositorio y se subió junto con un script	Desde entonces, cada cambio se sube en cuanto está terminado (ver apartado 8).

Versión 2

Error	Causa	Solución
El programa de `main` dejó de funcionar	El Pull Request del ticket se hizo contra `main` en vez de contra `version2` (PR #7), y el `main.py` de `main` todavía importaba `src`, que ya se llamaba `srcV1`	Lo dejamos así hasta pasar la versión 2 completa a `main`, que lo arregla. Desde entonces miramos siempre la rama de destino del PR.
`ticket2.py` no se podía importar	`calculos2.py` estaba vacío en todas las ramas	Hacer la clase `Calculadora`.
GitHub rechazó el push de `version2` («fetch first»)	Pau había subido la temporada mientras tanto	Hacer `git pull --no-rebase` de `version2` antes de subirla.
La prueba de la versión 1 fallaba	Importaba de `src` y esperaba 1000 pizzas para una temporada que no existe, cuando el código da un error	Importar de `srcV1` y comprobar que da `ValueError`. Añadir `tests/__init__.py` para lanzar todas las pruebas con un comando.
Git marcaba como modificados archivos que no habíamos tocado	Windows cambia los saltos de línea (CRLF)	Añadir solo los archivos de cada cambio, nunca `git add .`.
Al renombrar el log en VS Code salía en verde con una U	Git lo ve como un archivo borrado y otro nuevo sin guardar	Hacer `git add` de la carpeta: Git lo detecta como renombrado.
Un producto que no existía o una cantidad mal escrita solo daban un mensaje con `if`	No había excepciones	Excepciones propias (`ProductoNoEncontradoError`, `CantidadNoValidaError`…) capturadas en la caja.

7. Cambios de criterio respecto a la idea inicial

Antes	Después	Por qué
Carpetas `1.Main`, `2.Functions` y `3.SistemasDeOfertas`	Carpetas `src/`, `docs/` y `tests/`	Era más ordenado y evitaba añadir rutas a mano con `sys.path`.
8 pizzas	21 pizzas	El enunciado pide un mínimo de 20.
2x1 en Margarita, Prosciutto y Barbacoa	2x1 en Margarita, Prosciutto y Hawaiana, que se pueden mezclar	Es más justo para el cliente y más fácil de entender.
2 ofertas (2x1 y 10% en pedidos de más de 30 €)	4 ofertas: se añaden menú pizza + bebida y 20% en pizzas los martes	El enunciado pide al menos 3.
El programa preguntaba qué día era	Coge el día de hoy automáticamente	Era más cómodo; se puede cambiar desde el menú.

Antes	Después	Por qué
Ticket con columnas alineadas y fecha	Ticket sencillo: productos, descuentos, base imponible, IVA y total	Se parece más al ejemplo del enunciado y es más fácil de leer.

Versión 2

Antes	Después	Por qué
Un solo programa hecho con funciones	Dos versiones: la 1 con funciones y la 2 con clases; se elige al arrancar	Lo pide el proyecto y así se pueden comparar.
El carrito era una lista de nombres	`Carrito` con objetos `LineaCarrito` (producto + cantidad)	Si un producto ya está, se suma la cantidad en vez de repetirlo.
4 ofertas hechas con funciones	Clase base `Oferta` y 5 subclases, con una oferta nueva de temporada	Cada oferta se puede cambiar o añadir sin tocar las demás.
El menú pizza + bebida solo era una oferta automática	También se pueden pedir menús como producto, incluido el menú familiar	Lo pedía la tarjeta de Trello.
Gastos, margen y temporada en módulos con variables	Clases `Gastos`, `Margen`, `Temporada` y `Calculadora`	La calculadora recibe los objetos y se pueden probar otros valores.
Errores controlados con `if` y mensajes	Excepciones propias que heredan de `PizzeriaError`	Lo pedía la tarjeta de validaciones.
La tarjeta pedía la bebida con «precio de compra + margen»	Precio fijo de carta con el IVA incluido	Mismo criterio y mismos precios que en la versión 1.
Un log por cada rama	Un único resumen, `docsV2/Resumen_funciones_version2.md`	Es más fácil de leer.

8. Problemas de coordinación y cómo se solucionaron

1. **Dos versiones del proyecto a la vez.** Alex: estaba trabajaba con la estructura de carpetas antigua mientras que Pau reorganizaba el repositorio en `src/`. Lo solucionamos adaptando el código de Alex a la nueva estructura y manteniendo los nombres de funciones de Pau.
2. **Subida de las ramas de golpe.** Al pasar su parte al repositorio, Alex subió las cinco ramas (`feature/bebidas`, `feature/ticket`, `feature/menu`, `test/pruebas` y `docs/readme`) a la vez el 21/10, en lugar de una a una. Luego Pau las unió todas a `main` ese mismo día. No se puede deshacer porque GitHub guarda la actividad, así que lo explicamos aquí.
3. **Orden de los merges.** Algunas partes dependían de otras (el ticket necesita las ofertas y los cálculos). Acordamos un orden y comprobamos que el programa funcionaba después de unir todo.

Desde el 22/10 trabajamos así: `git pull` en `main`, una rama nueva por cada cambio, commits pequeños, `git push` al terminar y un merge para unirlo a `main`.

Versión 2

1. **Pull Request a la rama equivocada.** La rama del ticket se juntó con `main` en vez de con `version2`. Ahora comprobamos la rama base antes de crear cada PR.
2. **Subir `version2` a la vez.** Mientras uno subía su clase, el otro había subido la temporada y GitHub rechazó el push. Desde entonces hacemos `git pull` de `version2` justo antes de subirla.
3. **El tablero se quedó atrás.** Muchas clases estaban hechas, pero sus tarjetas seguían en «Pendiente». El 13/11 las clasificamos todas comparando su lista de pasos con el código.
4. **El código y el diseño de Trello no coincidían.** Al revisar las tarjetas vimos que pedían herencia, menús, `LineaCarrito` y excepciones propias que todavía no estaban. Las hicimos antes de cerrar la memoria.

En la versión 2 trabajamos así: `git pull` de `version2`, una rama nueva por cada clase o cambio, pruebas, commit, `git push` de la rama y merge con `version2`.

9. Cálculo económico: breve explicación del cálculo de precios

El precio de una pizza sale de sumar sus costes y aplicarle un 35% de beneficio y un 10% de IVA. Una Margarita cuesta 11,95 € en temporada media.

$$\text{precio} = \left(\text{ ingredientes} + 0{,}60 + \frac{4200 + 1600}{\text{pizzas al mes}}\right) \times 1{,}35 \times 1{,}10$$

1. **Ingredientes:** precio por kg × gramos / 1000, sumando todos los de la pizza.
2. **Suministros:** 0,60 € por pizza (luz, gas, agua, horno y embalaje).
3. **Gastos fijos:** personal (4200 €) + alquiler (1600 €) al mes, repartidos entre las pizzas que se esperan vender.
4. **Beneficio:** el coste total se multiplica por 1,35.
5. **IVA:** el precio sin IVA se multiplica por 1,10.

Desglose de la Margarita (250 g de harina, 100 g de tomate y 120 g de mozzarella) en temporada media:

Concepto	Importe (€)
Ingredientes	1,65
Suministros	0,60
Gastos fijos (5800 / 1000)	5,80
Coste total	**8,05**
Beneficio (35%)	2,82
Precio sin IVA	**10,87**
IVA (10%)	1,08
Precio de venta	**11,95**

Los gastos fijos pesan mucho, así que el precio cambia según la temporada:

Temporada	Pizzas al mes	Gastos fijos por pizza (€)	Precio de la Margarita (€)
Baja	600	9,67	17,70
Media	1000	5,80	11,95
Alta	1500	3,87	9,08

IVA en el ticket. Los precios de la carta ya llevan IVA. Al pagar se suman los productos, se restan los descuentos y del total se separa el IVA: base imponible = total / 1,10 e IVA = total − base imponible.

Bebidas. Tienen un precio fijo con IVA incluido (agua 1,20 €, refrescos 2,00 €, té frío 2,20 €, cerveza 2,50 €), porque no se cocinan.

Versión 2

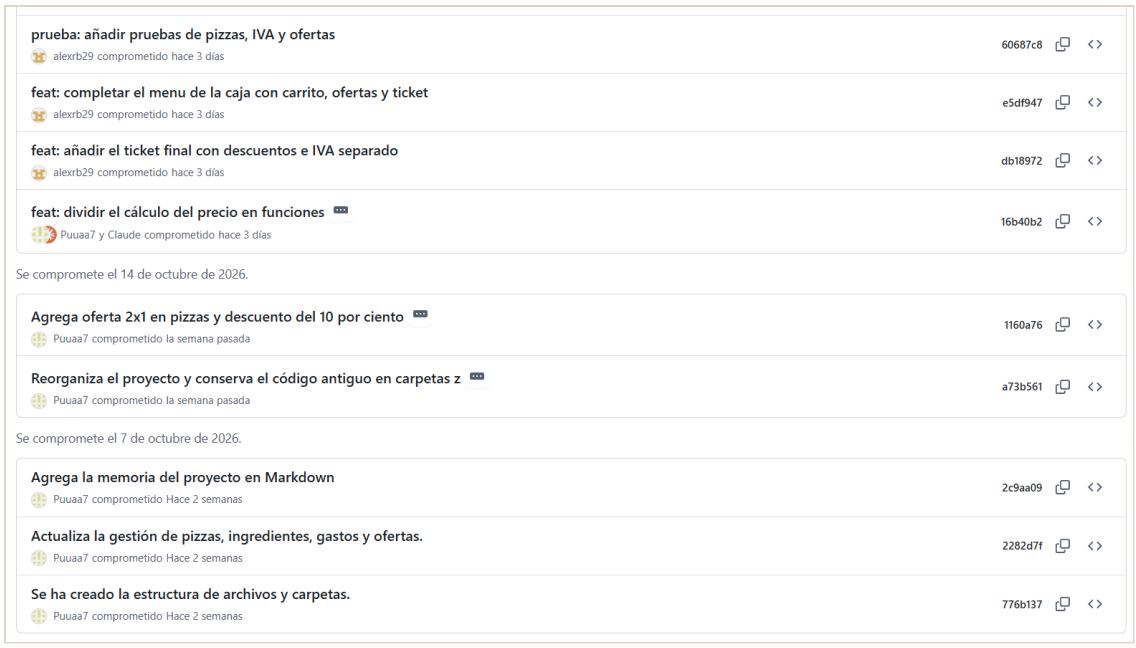
El cálculo es el mismo y da los mismos precios: la Margarita sigue costando 11,95 € en temporada media. Lo que cambia es quién lo hace:

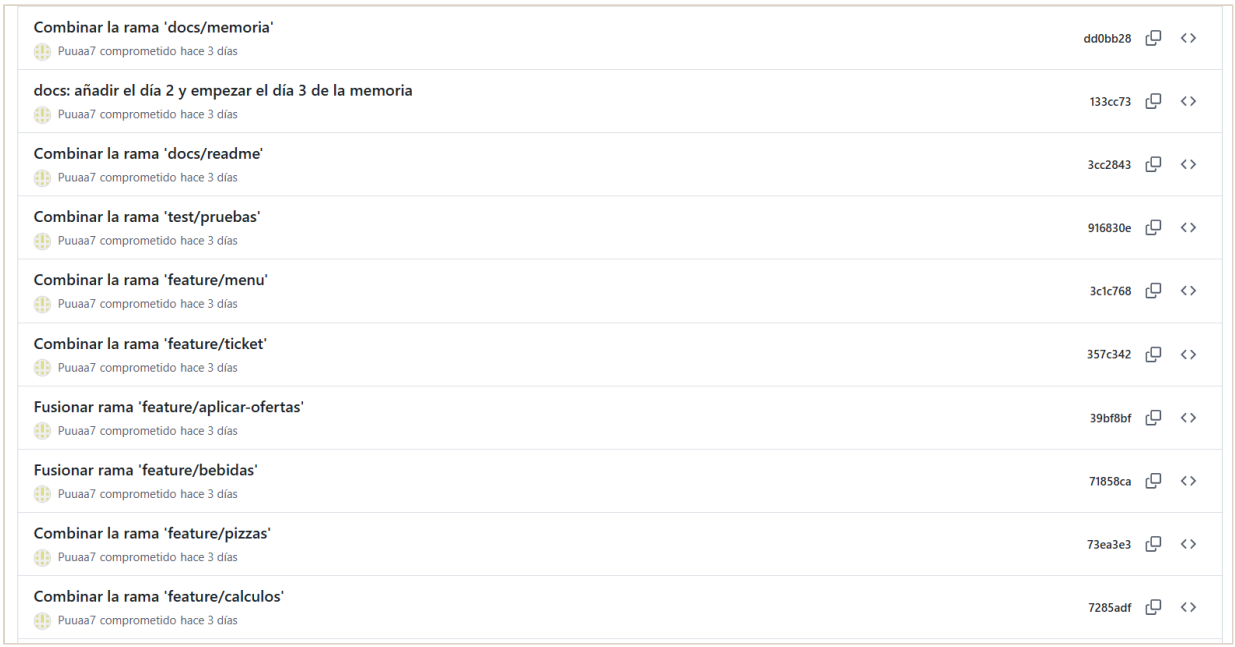
1. La `Pizza` suma el precio de sus objetos `Ingrediente` (`costeIngredientes()`).
2. La `Calculadora` añade los suministros y los gastos fijos, que coge de un objeto `Gastos`, y las pizzas al mes de la `Temporada` (`costeTotal()`).
3. La `Pizza` sobrescribe `precioSinIva()` de `Producto`: coste total × el `Margen` (1,35).
4. `Producto` pone el IVA igual para todos los productos: `precio()` = precio sin IVA × 1,10, redondeado a 2 decimales.

Menús. Un menú cuesta lo mismo que sus productos por separado menos su descuento. En temporada media, el menú Barbacoa + cola cuesta 14,12 + 2,00 − 1 = 15,12 €, y el menú familiar con Margarita, Funghi, agua y cola, 27,79 − 5 = 22,79 €.

10. Captura del historial de commits

El historial completo está en github.com/Puuaa7/ProyectoCajaRegistradora/commits/main. Cada parte del programa se hizo en su rama y se unió a `main` con un merge.


prueba: añadir pruebas de pizzas, IVA y ofertas
alexrb29 comprometido hace 3 días
60687c8
feat: completar el menu de la caja con carrito, ofertas y ticket
alexrb29 comprometido hace 3 días
e5df947
feat: añadir el ticket final con descuentos e IVA separado
alexrb29 comprometido hace 3 días
db18972
feat: dividir el cálculo del precio en funciones
Puuaa7 y Claude comprometido hace 3 días
16b40b2
Se compromete el 14 de octubre de 2026.
Agrega oferta 2x1 en pizzas y descuento del 10 por ciento
Puuaa7 comprometido la semana pasada
1160a76
Reorganiza el proyecto y conserva el código antiguo en carpetas z
Puuaa7 comprometido la semana pasada
a73b561
Se compromete el 7 de octubre de 2026.
Agrega la memoria del proyecto en Markdown
Puuaa7 comprometido Hace 2 semanas
2c9aa09
Actualiza la gestión de pizzas, ingredientes, gastos y ofertas.
Puuaa7 comprometido Hace 2 semanas
2282d7f
Se ha creado la estructura de archivos y carpetas.
Puuaa7 comprometido Hace 2 semanas
776b137


Combinar la rama 'docs/memoria'
Puuaa7 comprometido hace 3 días
dd0bb28
docs: añadir el día 2 y empezar el día 3 de la memoria
Puuaa7 comprometido hace 3 días
133cc73
Combinar la rama 'docs/readme'
Puuaa7 comprometido hace 3 días
3cc2843
Combinar la rama 'test/pruebas'
Puuaa7 comprometido hace 3 días
916830e
Combinar la rama 'feature/menu'
Puuaa7 comprometido hace 3 días
3c1c768
Combinar la rama 'feature/ticket'
Puuaa7 comprometido hace 3 días
357c342
Fusionar rama 'feature/aplicar-ofertas'
Puuaa7 comprometido hace 3 días
39bf8bf
Fusionar rama 'feature/bebidas'
Puuaa7 comprometido hace 3 días
71858ca
Combinar la rama 'feature/pizzas'
Puuaa7 comprometido hace 3 días
73ea3e3
Combinar la rama 'feature/calculos'
Puuaa7 comprometido hace 3 días
7285adf


Fusionar rama 'CalculoPorTemporada': validar cantidades de productos
Puuaa7 comprometido anteayer
2ab8baf
Validar cantidades antes de añadir pizzas y bebidas
Puuaa7 comprometido anteayer
b750d1d
Eliminar requirements.txt
alexrb29 autor anteayer
Verified
4046924
Combinar la rama 'CalculoPorTemporada'
Puuaa7 comprometido anteayer
62b5950
Validar temporadas exactas y sustituir match por if
Puuaa7 comprometido anteayer
c472bf1
Combinar la solicitud de extracción n.° 5 de Puuaa7/README
alexrb29 autor anteayer
Verified
3fbd357
docs: quitar la parte de ejecución del README
alexrb29 comprometido anteayer
2a7e4e1

Versión 2

El historial de la versión 2 está en github.com/Puuaa7/ProyectoCajaRegistradora/commits/version2. Cada clase tiene su commit (`feat: añadir clase Pizza (version 2)`, `feat: clase base Producto con herencia (version 2)`...) y cada rama se ha juntado con `version2`.

11. Conclusión

El programa cumple lo que pide el enunciado: 21 pizzas con su precio calculado, bebidas, carrito, 4 ofertas y un ticket con el IVA separado. Las 7 pruebas pasan correctamente.

Lo que mejor ha funcionado ha sido repartir las tareas en Trello por partes del programa, porque cada uno sabía qué le tocaba. Lo que peor ha funcionado ha sido Git: trabajamos demasiado tiempo fuera del repositorio y al final subimos y unimos muchas cosas a la vez. También durante una clase de proyecto trabajamos con live share en vez de con ramas, eso dificultó mucho hacer los comits por separado, solo los hacía Pau.

La lección principal es que hay que hacer commits y crear las ramas desde el primer día, y subir cada parte en cuanto está terminada. Arreglarlo al final nos costó más que hacerlo bien desde el principio.

Versión 2

La versión 2 hace lo mismo que la 1 con programación orientada a objetos: herencia (`Producto` y `Oferta` con sus subclases), clases abstractas, polimorfismo, encapsulación con `@property`, composición y excepciones propias. Además añade los menús como producto y una oferta de temporada. Pasan las 82 pruebas, que además comprueban que las dos versiones dan los mismos precios.

Lo que mejor ha funcionado ha sido hacer cada clase en su rama y juntarla con `version2` en cuanto estaba terminada: esta vez el historial muestra el trabajo día a día. Lo que peor ha funcionado ha sido no mover las tarjetas de Trello mientras trabajábamos: al revisarlas vimos que el diseño que habíamos acordado pedía cosas que el código aún no tenía.

La lección de la versión 2 es que el tablero tiene que ir al día con el código: si se compara cada tarjeta con lo que está hecho, se ve enseguida lo que falta.